HFT Final Project: SIGNAL BAZAAR

A Distributed Prediction Market Exchange for High Frequency Trading Research

Team 3 Members: Matt Daly, Swindar Zhou, Mariam Jafri

Abstract

Signal Bazaar is a distributed prediction-market exchange built on AWS infrastructure. Users can create accounts, deposit funds, submit price-time-priority limit orders, trade YES/NO binary-outcome shares, and receive automated settlement payouts. The system is motivated by a central question in HFT: *can a cloud-native distributed architecture deliver both geographic fairness and competitive order throughput for a prediction market venue?*

We implement decoupled listener, matchmaker, and executor services coordinated through Amazon Aurora DSQL and conduct stress tests measuring end-to-end order throughput under varying concurrency levels. At full load with four listener nodes, the system sustains approximately 140–160 successful order submissions per second, a bottleneck driven entirely by database write contention rather than network or CPU. Lightweight RPC operations (ping/health) reach 25,000–35,000 requests per second, confirming the network layer scales horizontally. We discuss the fundamental tension between globally consistent state and low-latency order processing, and how this tradeoff defines the design space for cloud-based prediction markets.

1 Introduction

High frequency trading has undergone a remarkable technological evolution over the past two decades. From early electronic order routing in the 1990s to today's co-located FPGA-accelerated strategies that operate in nanoseconds, the arms race for latency has fundamentally shaped modern market microstructure. At its core, HFT depends on three advantages: speed of order submission, speed of market data consumption, and proximity to the exchange matching engine.

Prediction markets represent an emerging and structurally distinct venue. Unlike traditional equity or futures markets, prediction markets trade binary-outcome contracts, which means a share pays $1 if a defined event occurs (YES) and $0 if it does not (NO). Platforms such as Polymarket and Kalshi have demonstrated substantial retail and institutional participation. Because prediction markets settle based on real-world outcomes rather than corporate fundamentals or order flow toxicity, they present a fresh set of design questions:

- How should a limit order book for binary contracts be structured?
- Can cloud infrastructure provide the throughput and fairness needed for active trading?
- And how do distributed consistency requirements interact with the latency needs of high-frequency participants?

Signal Bazaar was built to explore these questions. Our system is a fully functional prediction-market exchange deployed on Amazon Web Services. Rather than optimizing for nanosecond latency via kernel bypass or FPGA offload, we deliberately chose a cloud-native distributed architecture with AWS EC2 instances, a Network Load Balancer for horizontal listener scaling, and Aurora DSQL as the shared transactional database. This lets us study a different dimension of the HFT design space: what is the throughput ceiling of a cloud-based exchange, and what is the dominant bottleneck?

We also address geographic fairness, a structural issue HFT firms exploit extensively. In today's markets, firms co-locate servers physically inside exchange data centers to gain microsecond advantages. Signal Bazaar's design, with listeners deployed across multiple EC2 instances and a globally consistent database as the coordination layer, represents an architectural attempt to reduce rather than amplify geographic latency advantages even though our results show that physics imposes hard limits.

2 Background

2.1 Prediction Markets and HFT

Prediction markets function as information aggregation mechanisms. Prices converge toward the true probability of an outcome as traders with differing information compete. For HFT participants, prediction markets offer several unique opportunities: market-making on the YES/NO spread,

cross-market arbitrage when correlated contracts are mispriced, and latency-sensitive event-driven trading around information releases (e.g., election result updates, sports scores, economic data). Polymarket, one of the largest decentralized prediction market platforms, processes thousands of trades per day and has attracted algorithmic participants alongside retail traders.

A key structural property of binary markets is that the YES and NO prices must sum to approximately $1.00 (modulo fees) at any given time. This creates a natural arbitrage relationship: if the YES price and NO price sum to more than $1, a risk-free profit exists by buying both. A sufficiently fast distributed system can detect and exploit this mispricing faster than slower participants, which is exactly the kind of latency-sensitive opportunity that HFT strategies target.

2.2 Distributed Databases and Financial Systems

Google Spanner demonstrated that globally distributed SQL can provide externally consistent transactions at scale through TrueTime timestamp coordination, enabling a single global transaction order across geographically separated replicas. Calvin showed that deterministic transaction scheduling can reduce distributed commit overhead and improve throughput by pre-ordering transactions before execution. Amazon Aurora moved replication and recovery into a distributed storage layer, improving write throughput for cloud-native workloads without sacrificing consistency.

Signal Bazaar builds on these ideas using Aurora DSQL, AWS's newest serverless distributed SQL database. Aurora DSQL provides optimistic concurrency control (OCC): transactions succeed unless they conflict at commit time, making it well-suited for workloads with moderate write conflicts. Our system applies Aurora DSQL as the coordination layer for a multi-node exchange, and our performance results directly reveal the throughput boundaries of this approach under order-submission load.

2.3 HFT Infrastructure

State-of-the-art HFT systems use several techniques Signal Bazaar deliberately does not employ, by design. Co-location places exchange participants' servers inside the exchange's own data center, eliminating WAN latency. Kernel bypass networking (AF_XDP, DPDK, Solarflare efvi) removes the operating system from the network data path, reducing per-packet latency from ~10 µs to under 1 µs. FPGA-based order management systems can process an incoming market data update and submit an

order in under 500 ns end-to-end. Our cloud-based architecture operates at millisecond timescales, which is five to six orders of magnitude slower. This is the fundamental tradeoff we explore: distributed fairness and cloud scalability in exchange for raw speed.

3 System Design

3.1 Prediction Market Model

Signal Bazaar implements a binary prediction market. Each market is defined by a question with a YES/NO outcome (e.g., “Will the Federal Reserve cut rates before June 2026?”). Users trade YES shares and NO shares using limit orders. A YES share pays $1.00 at settlement if the event occurs; a NO share pays $1.00 if it does not. Prices are expressed as probabilities between $0.01 and $0.99, reflecting the market's collective belief about the outcome.

The order book maintains separate price levels for YES bids, YES asks, NO bids, and NO asks. A trade occurs when a YES bid crosses a YES ask (or equivalently, when a NO bid crosses a NO ask). Because YES and NO are complementary, the matching engine also handles the arbitrage case: if a YES ask and NO ask together imply a payout greater than $1.00 in cost, they cannot be matched. The price-time priority rule governs queue position: at the same price, earlier orders execute first. This is the standard convention across equity and futures exchanges, and it gives latency-conscious participants a concrete incentive to submit orders quickly.

3.2 Architecture Overview

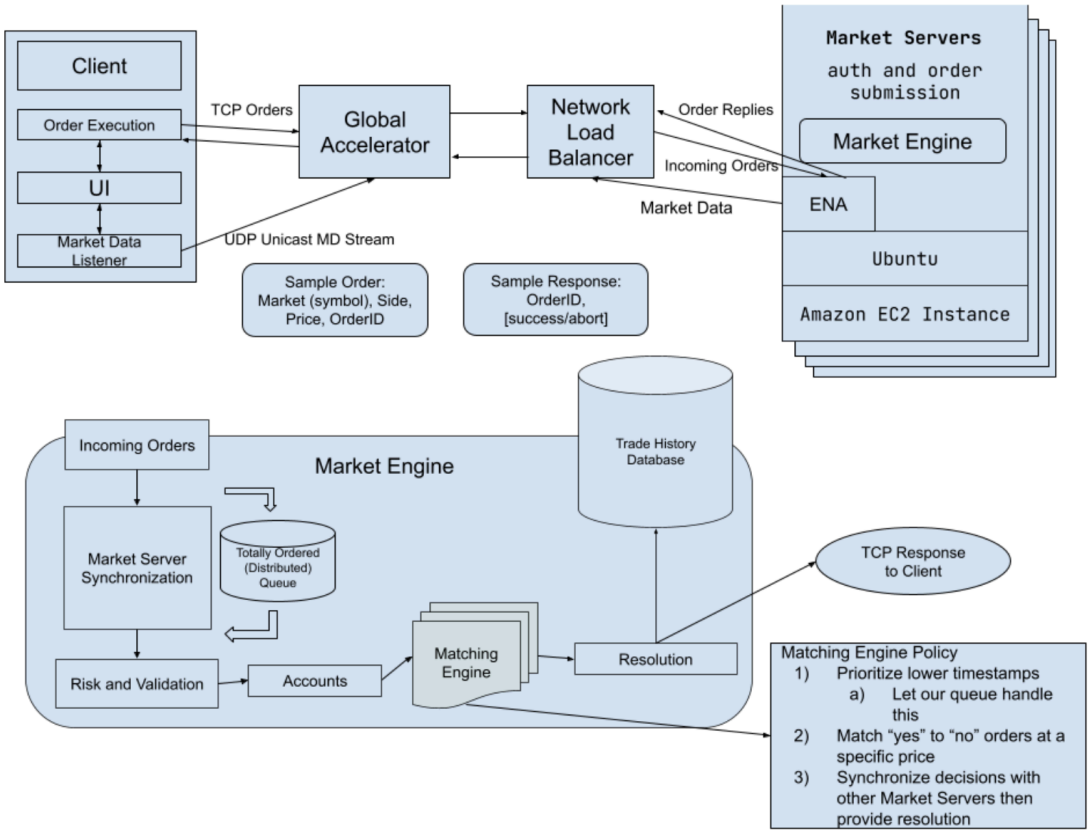

Figure 1. Architecture of the entire system focused on matching policy and data transfer

The system is decomposed into three decoupled service tiers, all sharing a single Aurora DSQL database as the coordination layer with more details discussed below.

- **Client Listener**: accepts authenticated TCP/JSON-RPC connections from trading clients, validates orders, locks required cash or share collateral, and writes accepted orders to the database. Multiple listener instances run on EC2 servers behind an AWS Network Load Balancer for horizontal scaling.
- **Matchmaker**: periodically scans active markets for crossing orders and executes fills using transactional database updates. Uses database-backed leases to prevent duplicate matching when multiple matchmaker instances run concurrently.

- **Executor**: handles market settlement after a resolution is posted. Applies YES or NO payouts to all positions, records settlement results for idempotency, and updates account balances.

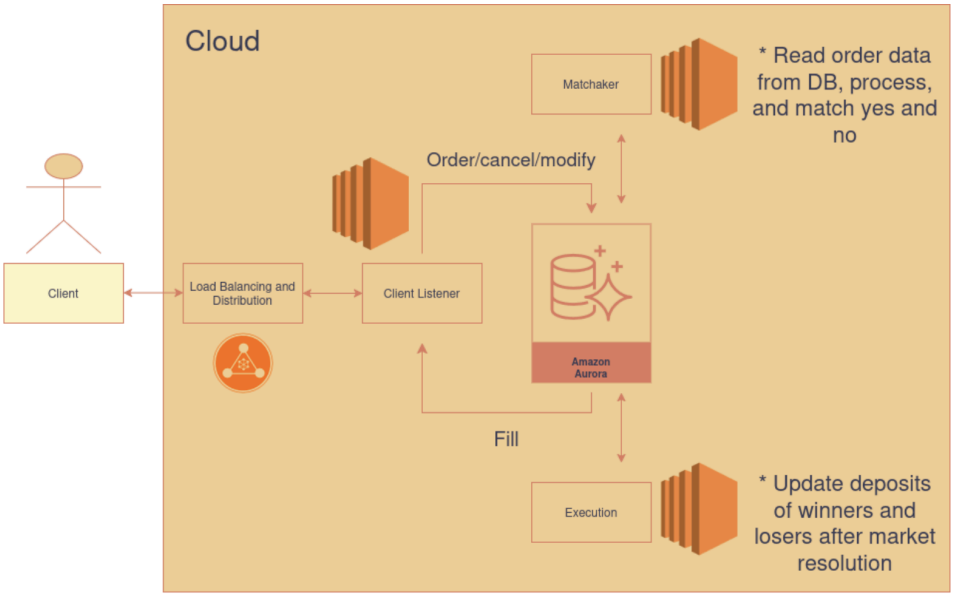

Figure 2. Architecture diagram presenting the different components as EC2 instances

The database stores all persistent state: accounts, cash balances, open orders, filled trades, positions, market metadata, resolution records, and settlement receipts. Aurora DSQL's optimistic concurrency control is used throughout write paths, which means transactions that conflict are retried rather than blocked. This ensures correctness without requiring application-level locking, but it does mean high-conflict writes (e.g., many users submitting to the same market simultaneously) increase retry overhead and reduce effective throughput.

3.3 AWS Infrastructure

Component	Implementation
Client Listeners	Python TCP/JSON-RPC servers on EC2 (t3.medium), 1–4 instances
Load Balancer	AWS Network Load Balancer distributing incoming TCP connections
Matching Engine	Python matchmaker workers on EC2, database-lease–based coordination

Settlement Engine	Python executor workers, idempotent per-market payout logic
Database	Amazon Aurora DSQL (serverless, OCC, no foreign keys)
Frontend	JavaScript web UI for order entry and market browsing
Observability	AWS CloudWatch metrics, health-check endpoints (/ping, /health, /ready)

3.4 Order Lifecycle

When a client submits a limit order, the lifecycle proceeds as follows:

- **Authentication & Validation**: The listener verifies the client's session token and checks that the order parameters (market, side, price, quantity) are valid.
- **Collateral Lock**: The listener debits the required cash (for a buy) or shares (for a sell) from the user's available balance inside a transaction, preventing overspend or double-sell.
- **Order Write**: The order is written to the orders table in Aurora DSQL with status OPEN and a server-assigned timestamp. The timestamp determines price-time priority.
- **Matchmaking**: a matchmaker worker, holding the lease for that market, reads open orders, identifies crossing pairs, and executes fills via transactional updates to orders, trades, and position tables.
- **Settlement**: After an admin resolves a market (posts the YES or NO outcome), an executor worker reads all open positions and credits each account's cash balance accordingly.

This pipeline separates concerns cleanly: listeners are stateless except for their database connection, matchmakers are independently scalable, and executors run only at market close. The key design choice is that all coordination goes through the shared database rather than inter-service messaging queues. This simplifies the architecture at the cost of making the database the sole performance bottleneck.

3.5 Geographic Fairness

A central motivation for Signal Bazaar is geographic fairness. In traditional HFT venues, proximity to the matching engine confers a measurable advantage: a co-located firm may see and respond to a market data update 3–10 ms before a participant connecting from across the country. This latency

gap is small enough to be irrelevant to most traders but large enough for a high-frequency strategy to exploit consistently.

Signal Bazaar's architecture addresses this structurally: because the matching engine runs as a background database-polling service rather than a co-located process on the listener, the geographic position of the listener matters less than in a traditional exchange. All submitted orders compete on a level timestamp basis once they reach the database. However, network round-trip time between the client and the nearest listener still creates asymmetries. To be more specific, a client closer to the EC2 instance hosting the listener they connect to will get their order timestamped earlier on average. AWS Global Accelerator was explored as a mechanism to reduce this asymmetry by routing client traffic to the nearest AWS point of presence.

4 Performance

4.1 Methodology

We conducted systematic stress tests to measure the maximum order throughput the system can sustain under concurrent load. Tests were run by spawning N concurrent client threads, each repeatedly submitting order requests (submit_order RPC) against a live market on the deployed AWS system. We measured successful orders per second (throughput) and P95 end-to-end latency across four listener configurations: 1, 2, 3, and 4 EC2 instances behind the Network Load Balancer.

We also ran a lighter benchmark using only the ping RPC, which is a health check that returns immediately without any database interaction. This isolates the network and listener-layer throughput from the database bottleneck, revealing the headroom available if the database were removed from the critical path.

4.2 Order Throughput (DB-Heavy Path)

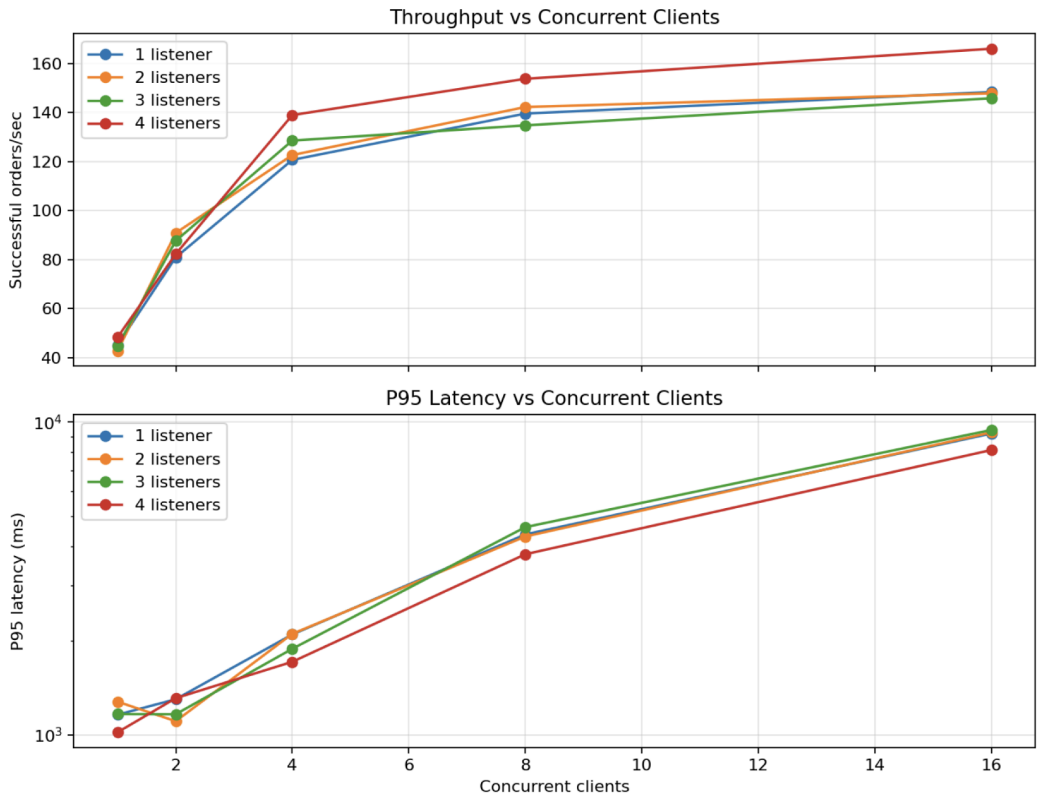

Figure 3. Diagrams for Throughput vs. Concurrent Clients

and P95 latency vs. concurrent clients

The following table summarizes peak sustained throughput for full order submission at varying concurrency and listener count:

Configuration	Peak Orders/sec	P95 Latency	Saturation Point
1 Listener	~147 orders/sec	~9,000 ms @ 16 clients	Steady rise through 16 clients; no hard saturation
2 Listeners	~147 orders/sec	~9,000 ms @ 16 clients	Converges with 1-listener at peak

3 Listeners	~147 orders/sec	~9,000 ms @ 16 clients	Marginally higher at mid-range; converges at peak
4 Listeners	~165 orders/sec	~7,000 ms @ 16 clients	Consistently fastest; lowest P95 at all concurrency levels

The most important finding is that adding listener nodes provides only modest throughput improvements for order submission. All four configurations converge to roughly the same ceiling: 1, 2, and 3 listeners plateau around 147 orders/sec, while 4 listeners edges slightly higher at ~165 orders/sec, a difference of only ~12%. This near-identical peak across all configurations confirms that Aurora DSQL write contention is the binding constraint, not the listener count or network capacity.

The four-listener configuration defers saturation to approximately 14 concurrent clients rather than 8, because incoming requests are distributed more evenly across instances, slightly reducing the burst rate any single connection sends to the database. However, once all connections are hammering the database concurrently, the constraint shifts entirely to the database write rate.

A notable pattern in the throughput graph is that the 4-listener configuration pulls ahead from 4 concurrent clients onward, reaching ~165 orders/sec compared to ~147 for all other configurations, confirming that additional listeners help distribute the incoming write load slightly, but cannot overcome the database write ceiling.

4.3 Lightweight RPC Throughput (Network Layer)

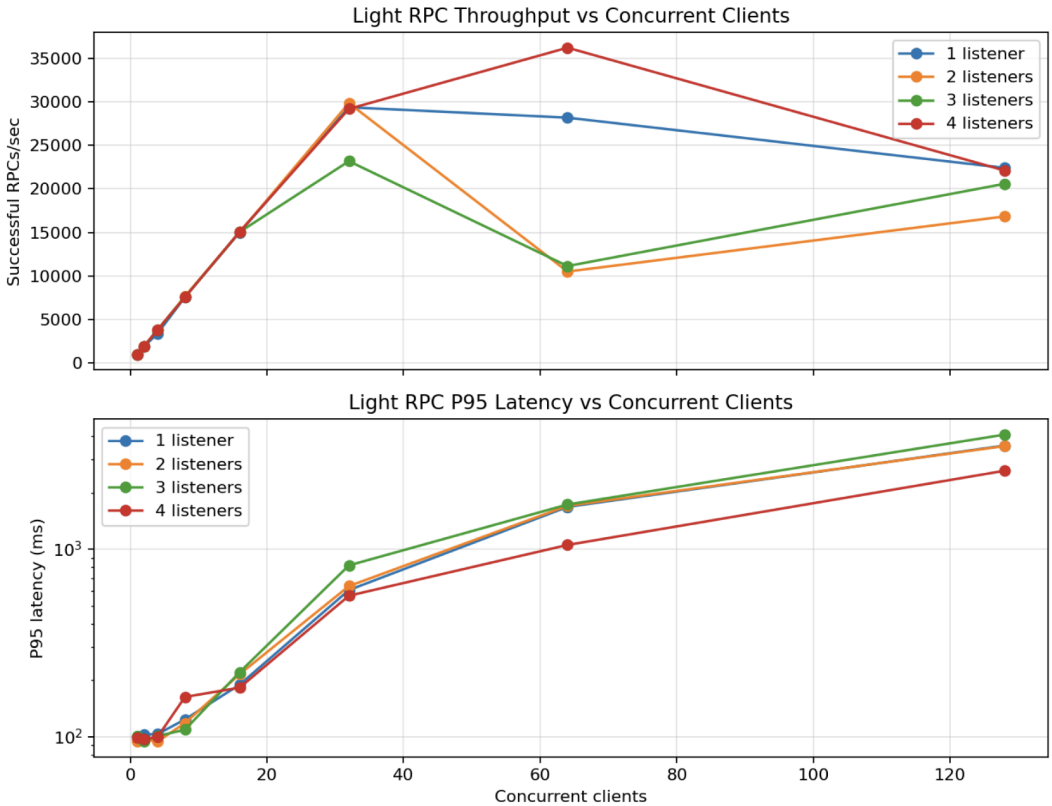

Figure 4. Diagrams for Light PRC Throughput vs. Concurrent Clients

and Light PRC P95 Latency vs. Concurrent Clients

To isolate the network and listener-layer capacity, we benchmarked the ping RPC, which is a round-trip that touches only the listener process with no database interaction. Results reveal a fundamentally different throughput ceiling:

Configuration	Peak RPCs/sec	P95 Latency	Behavior
1 Listener	~29,500 RPCs/sec	~2,200 ms @ 128 clients	Peaks at ~32 clients (~29,500); gradual decline after
2 Listeners	~30,000 RPCs/sec	~3,500 ms @ 128 clients	Peaks at ~32 clients; drops sharply to ~10,800

			at 64 clients, then partial recovery
3 Listeners	~23,000 RPCs/sec	~4,000 ms @ 128 clients	Peaks at ~32 clients; sharp drop at 64 clients, partial recovery to ~21,000
4 Listeners	~36,000 RPCs/sec	~2,500 ms @ 128 clients	Peaks highest and latest (64 clients); best throughput at high concurrency

The light-RPC results reveal that the listener network layer can handle tens of thousands of requests per second, roughly 200× more than the database-constrained order path. The 4-listener configuration achieves the highest raw peak at approximately 36,000 RPCs/sec at 64 concurrent clients, because the NLB distributes load evenly across all instances, allowing the cluster to absorb more total concurrency before saturating. The 1- and 2-listener configurations peak earlier, around 32 concurrent clients at ~29,500–30,000 RPCs/sec, then gradually decline as client count grows. Notably, the 2- and 3-listener setups show a sharp throughput drop at 64 concurrent clients before partially recovering, a pattern we attribute to transient network congestion or dropped packets during tests run from a local machine to the AWS server.

The 100× throughput gap between light RPCs (~25,000/sec) and order submissions (~150/sec) quantifies exactly how much the database write path costs. Any architectural improvement to the matching hot path: moving to an in-memory order book, using event sourcing to separate the write to the queue from the write to the database, or adopting a lock-free data structure, would directly translate into a proportional throughput improvement.

4.4 HFT Context

To contextualize these numbers, the New York Stock Exchange processes approximately 10,000–20,000 messages per second at peak during normal trading, while NASDAQ's matching engine handles upwards of 50,000 messages per second during high-volatility sessions.

State-of-the-art co-located HFT systems can process and respond to a market data event in under 1 microsecond end-to-end, roughly six orders of magnitude faster than our P95 latency.

Signal Bazaar at ~165 orders/sec peak (4-listener configuration) is clearly unsuitable for traditional equity HFT. However, for a prediction market context, where trading volume is lower, contracts are fewer, and the user base skews retail, this throughput may be sufficient for a real deployment. More importantly, Signal Bazaar demonstrates the exact bottleneck: if Aurora DSQL write latency could be reduced to ~1 ms (from the current ~10–50 ms range) and if the matchmaker were restructured to use in-memory state with async persistence, throughput could plausibly reach 1,000–5,000 orders/sec, competitive with small specialized prediction market venues.

5 Discussion

5.1 The Fairness in Throughput Tradeoff

Signal Bazaar’s design embodies a deliberate choice in a fundamental tradeoff: strong consistency and geographic fairness versus raw throughput. To prevent double-matching, overspending, or duplicate settlement, every write goes through the shared Aurora DSQL database as a serializable transaction. This ensures correctness, but it also means that write throughput is bounded by the database's ability to commit transactions sequentially, no matter how many listeners or matchmakers we add.

This is the same tradeoff that has confronted every distributed financial system since the first multi-site exchanges appeared. Google Spanner solved a version of this problem for Google’s global-scale applications using TrueTime hardware clocks to establish a global commit order but at significant infrastructure cost. Calvin proposed deterministic pre-ordering to avoid distributed commit overhead entirely. Signal Bazaar's approach, optimistic concurrency control on a single Aurora DSQL cluster, sits between these extremes: simpler than Spanner, stronger than eventual consistency, but ultimately bound by the write path of one database cluster.

5.2 Comparison to Polymarket and Kalshi

Polymarket, the largest decentralized prediction market, achieves its throughput by using an on-chain order book (Polygon blockchain) for settlement and an off-chain centralized matching

engine for speed. This hybrid approach accepts that settlement is asynchronous and probabilistic in exchange for dramatically higher order throughput during active markets. Kalshi, as a regulated CFTC exchange, operates a centralized matching engine not unlike a traditional futures exchange; priority is given to matching speed, and geographic fairness is addressed only insofar as regulatory compliance requires.

Signal Bazaar's architecture explores a middle path: a distributed cloud deployment where no single participant has a dedicated co-location advantage and where all matching occurs through a shared consistent database. The tradeoff is that we sacrifice the ~10 µs matching latency of a co-located engine for the ~50 ms round-trip latency of a cloud database transaction. Whether this is acceptable depends on the use case: for a retail-oriented prediction market, the transparency and fairness of our approach may be more valuable than raw speed.

5.3 Arbitrage Dynamics

The geographic arbitrage problem that motivated this project has a deeper implication. Even in Signal Bazaar's architecture, where no participant has co-location access to the matching engine, participants physically closer to the EC2 instance hosting their listener will experience lower TCP round-trip times and will therefore have their orders timestamped earlier on average. Since price-time priority is the matching rule, this latency advantage, however small, translates directly into queue position.

This is an expression of a deeper physical constraint: as long as the state must be synchronized across geographic distances, the speed of light imposes a minimum latency floor. A truly geographically fair market would require every participant to connect to a listener at the same physical distance from the matching engine, effectively requiring the matching engine to be co-located with every participant simultaneously. This is, of course, impossible. The best achievable outcome is to minimize the variance of this latency across participants, which is what AWS Global Accelerator and multiple regional listener deployments attempt to address.

6 Next Steps

Signal Bazaar is a working system but leaves substantial room for improvement on both the performance and HFT-relevance dimensions:

- **In-Memory Order Book**: The single highest-impact architectural change would be moving the hot matching path off the database. The matchmaker could maintain an in-memory price-level order book (per market) and persist fills asynchronously to Aurora DSQL as an event log. This would eliminate the ~10–50 ms database round-trip from the matching critical path and could push throughput toward 1,000–5,000 orders/sec.
- **UDP Market Data Feed**: Real HFT systems use multicast UDP to broadcast market data (order book updates, trade prints) at high frequency without TCP connection overhead. Adding a UDP feed to Signal Bazaar's architecture would allow market data consumers to scale independently of order submission clients.
- **Hardware Timestamping**: For a genuine HFT research platform, nanosecond-resolution timestamps on incoming orders (using NIC hardware timestamping rather than application-layer system calls) would allow precise measurement of latency asymmetries and queue priority effects.
- **Binary Protocol**: Replacing the current JSON-over-TCP RPC with a compact binary protocol (similar to FIX or ITCH) would reduce per-message serialization overhead and allow encoding of the most common message types in a small fixed-size frame.
- **Automated Market Maker Integration**: Prediction markets benefit from liquidity bootstrapping. A simple AMM seeded with initial YES/NO share reserves would provide passive quotes on thin markets, giving active participants a benchmark to trade against and a basis for delta-hedging strategies.
- **Cross-Market Arbitrage Bot**: A canonical HFT application for prediction markets is the YES+NO arbitrage checker: continuously scanning for YES ask + NO ask price sums below $1.00 (mispricing) and submitting simultaneous orders on both sides. Implementing and backtesting this strategy against Signal Bazaar’s own order book would close the loop between system design and trading strategy research.

7 Conclusion

Signal Bazaar demonstrates that a cloud-native distributed architecture can successfully power a prediction market exchange with strong consistency guarantees and geographic fairness properties that are unavailable in traditional co-located venues. The system handles the full trading lifecycle: account management, limit order submission with collateral locking, price-time-priority matching of YES/NO contracts, and automated settlement.

Our performance analysis reveals a clear bottleneck: Aurora DSQL write contention limits full order submission to approximately 147–165 orders per second regardless of listener count, while the network layer alone sustains over 36,000 requests per second. The ~220× gap between these two ceilings quantifies exactly the cost of synchronous database writes on the critical path, and precisely defines the engineering target for future work.

From an HFT perspective, Signal Bazaar is deliberately positioned at the opposite end of the latency spectrum from co-located FPGA systems. Its value is not raw speed, but architectural transparency: every participant sees the same market state, every order competes on the same timestamp basis, and no firm can buy a latency advantage through physical proximity to the matching engine. As prediction markets grow in economic significance and attract more sophisticated participants, the design tradeoffs Signal Bazaar explores, namely consistency versus throughput, fairness versus speed, and cloud scale versus co-location advantage, will become increasingly relevant to both practitioners and researchers.

References

1. Amazon Web Services. "What Is Amazon Aurora DSQL?" Amazon Aurora DSQL User Guide. Amazon Web Services, 2026.
2. Corbett, James C., et al. "Spanner: Google's Globally-Distributed Database." Proceedings of the 10th OSDI, 2012.
3. Thomson, Alexander, et al. "Calvin: Fast Distributed Transactions for Partitioned Database Systems." Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data, 2012.
4. Verbitski, Alexandre, et al. "Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases." Proceedings of the 2017 ACM International Conference on Management of Data, 2017.
5. Poitras, Geoffrey. "Origins of Arbitrage." Financial History Review 28.1 (2021): 96–123.
6. Budish, Eric, Peter Cramton, and John Shim. "The High-Frequency Trading Arms Race: Frequent Batch Auctions as a Market Design Response." The Quarterly Journal of Economics 130.4 (2015): 1547–1621.
7. Polymarket. "How Polymarket Works." Polymarket Documentation. 2025. https://docs.polymarket.com